Concurrency Issues in C^{++}

Eshrat Arjomandi, York University
and
William O'Farrell, IBM Canada

Abstract

In this paper we review and analyze concurrency issues in object-oriented languages. We particularly focus on C^{++}-based languages. A categorization of existing concurrent object-oriented languages according to their level of integration between the paradigms of object-oriented programming and concurrent programming is presented. We then review concurrent C^{++}-based languages. An analysis of concurrency issues in C^{++} is also presented.

1 Introduction

The notion of an object was first introduced in Simula [14] and became popular with the introduction of Smalltalk-80 [17]. It is commonly accepted that a language is classified as object-oriented if it supports objects (a *self-contained* entity supporting a set of operations), classes (objects with the same behaviour), encapsulation (communicating with an object only via its externally visible operations), and code sharing (inheritance, delegation, and so on). For a more detailed discussion of object-oriented programming (OOP), the reader is referred to [13, 18, 23, 26, 27].

Concurrency has been the subject of much research for over three decades. Traditionally, concurrency is concerned with threads of control and problems of synchronization and mutual exclusion. OOP languages provide a natural facility for modelling and decomposing a problem into *self-contained* entities called objects. An object is self-contained in the sense that it has exclusive control over its own internal state and communicates with the outside world through message passing. Hence it appears that OOP presents an ideal environment for concurrent programming. OOP provides the ground work for modelling a real-world problem and exploiting its natural concurrency without much concern for synchronization and mutual exclusion.

Two approaches are used in applying the facilities of OOP to ease the problems of concurrent programming. In the first approach, language extensions or new concurrent languages are introduced. These languages are used in application areas such as real-time systems, artificial intelligence, systems, and symbolic manipulation. The second approach uses OOP in developing a library of reusable abstractions that encapsulate the lower level details of concurrency (for example, architecture, data partitions, communication and synchronization) [25]. In particular, application-oriented libraries allow for efficient parallel computation, at least for a specific range of applications. Such libraries can be coded and tuned to make best use of the hardware resources, even for fine-grained applications such as n-body calcu-

The IBM contact for this paper is William O'Farrell, Centre for Advanced Studies, IBM Canada Ltd., Dept. B2/894, 895 Don Mills Road, North York, Ontario M3C 1W3

lations [9]. This approach is suitable for a variety of scientific computation applications. It should be noted that the two approaches – language extensions and libraries – are not completely disjoint. In particular, C^{++} [30] class libraries can be designed to add concurrency to objects via the inheritance mechanism. This approach was taken in the parallelization of a popular co-routine library [15]; here, objects that inherit from a base class called *task* become active (i.e. are given a thread) through the action of an architecture-specific constructor function.

In this paper, we focus on concurrency issues in C^{++}. Section 2 overviews different models of concurrency. Section 3 classifies some of the existing concurrent OOP languages and gives a brief overview of these languages. Section 4 provides a review and an analysis of C^{++}-based languages. Concluding remarks and research issues are covered in Section 5.

2 Models of Concurrency

In this section we discuss models of concurrent execution and communication that are commonly found in OOP languages.

Majority of concurrent OOP languages assume a parallel model consisting of a set of processors and a communication subsystem. One of the most important aspects of a concurrency model is its memory model. Most concurrent languages can be assigned to one of two camps: distributed memory or shared memory. In a distributed memory model, processors have exclusive access to their own local memories. Processors communicate via explicit messages, carried by a communication subsystem. In most distributed memory machines, the message passing subsystem hides the actual underlying topology from the user, and allows messages to be addressed to arbitrary processors (although with varying latency). We shall assume this capability.

The shared memory model of parallel computing has enjoyed continued popularity for more than two decades. In a shared memory model, all processors share a large global memory, or at least so it appears to the programmer. In reality, nearly all such machines have some aspect of memory distribution, even if only local caches. Such machines are thus said to be *NUMA* – non-uniform memory architecture. In fact, the actual machine may be something close to a true distributed memory machine, but with hardware and software to present the illusion of shared memory to the user. This variety of architecture is often referred to as *Distributed Shared Memory* [31]. Shared memory can even be simulated on collections of autonomous computers connected via a network. For instance, Orca[7] is a distributed shared memory programming language employing shared objects.

In discussing concurrency, a distinction should be made between parallel programming and distributed systems. The latter term is usually used to describe systems in which the distribution of data and control is fundamental to the function of the system; that is distribution is not only an alternative model for achieving concurrency, but rather is axiomatic to the conception of the system. In distributed systems, concurrency is often secondary to such issues as fault detection and recovery, functional distribution, security of data access, etc. By contrast, parallel programming is almost always concerned with the degree of concurrency, and the means to express it. Typically, concurrent programming languages assume reliable hardware, expect automatic or semi-automatic load balancing, leave security to the operating system, etc. In this paper we are specifically concerned with parallel programming, not distributed systems, although there is clearly an overlap between the two areas. For a survey of distributed programming languages, see [6]. The rest of this section is devoted to discussing issues in three paradigms of parallel programming: shared memory, distributed memory, and message passing. For more detail on these issues outside of the context of object-oriented programming, see [4].

Issues in Shared Memory. In a shared memory parallel computer (also sometimes called a multicomputer, or a symmetric multiprocessor), the logical view of the system that is presented to the user is that of all processors reading and writing to a single shared global address space. Processes (that is, threads of

control that are run on processors) usually communicate through the use of shared variables. If process A updates variable X, then process Y immediately sees the update. Basic operations, such as update of a memory word, are always considered indivisible so that Y cannot accidentally get part of the old X and part of the new X – it gets either all of the old or all of the new. Because data structures are often composite, some locking mechanism is usually provided to enable processes to obtain temporary exclusive access to a multi-word region of memory. Whatever mechanism is provided by the hardware, programming languages usually provide higher-level constructs (which are implemented using the machine-level mechanisms) to make region locking convenient.

One of the more popular constructs for mutual exclusion is that of a *monitor*. In its original formulation, a monitor was an abstract data-type comprising static data and a set of operations to be applied to that data. When a process invokes a monitor operation, the system ensures that the operation does not commence until exclusive access to the monitor data is guaranteed. Thus, a process can be blocked while waiting for another that entered the monitor ahead of it to complete its operation. Monitors usually provide condition queues – a mechanism to allow a process to suspend itself inside of a monitor operation while waiting for some condition to be met. When suspended, a process no longer blocks others waiting to enter the monitor. Because they are based on the notion of abstract data type, monitors fit very naturally into the framework of concurrent object-oriented programming languages.

Issues in Distributed Memory. In distributed memory machines, each process has access only to its own memory, and communicates with other processes via explicit messages. Because they do not have the overhead associated with maintaining a consistent view of global memory, distributed memory architectures tend to be more scalable than shared memory architectures. In most such machines, the program (which is constant throughout execution) is replicated on each processor. Such systems are often referred to as SPMD – single program, multiple data. Each processor can execute its own copy of the program independently of the other processors, and may, in fact, have several processes that it time-slices between. It may even be a fully functional shared memory machine in its own right, and be capable of running multiple threads concurrently. If, however, processes are to be capable of being migrated from processor to processor, they must be programmed as if they were isolated from the memory space of all other processes, so that migration doesn't unexpectedly change their behavior. In an object-oriented system having processes bound to objects (that is, active objects) implies that objects must be fully encapsulated.

Although each processor is operating independently, it nonetheless is cooperating with the other processors in the machine on a single parallel computation. It will thus frequently need to communicate: that is, to send and receive messages. This function is the responsibility of the machine's message-passing subsystem. The subsystem may be implemented entirely in hardware, or may comprise a combination of hardware and software, but it is usually (although not always) considered to operate autonomously. Depending on the message-passing style chosen, processes may or may not be blocked on either message send or message receive operations – we discuss this issue below. At the lowest level, a message destination address is a specific processor, so if the message is to be delivered to one of several processes, information about the destination process must be included in the message. Also, information on the originating processor and process is usually available as well, and some systems allow a processor to block until a message from a specific source is received. In concurrent programming languages that support message passing at the user level, details of message addressing are usually abstracted away, enabling the programmer to address messages to named transactions or channels.

Issues in Message Passing. Communications in distributed memory computers is via message passing. Message passing, however, can be applied in both distributed and shared memory environments. Object-oriented pro-

gramming is often regarded as a form of programming with messages, even on sequential machines. Specifically, invoking a particular method of a particular object can be seen as sending a message to the object, the contents of the message being the method name and its parameters. Nevertheless, because of the close analogy to message passing, object-oriented languages would seem to fit naturally into the distributed memory model. However, if that is to be so, the language must forbid messages to contain references or pointers to data in the address space of the sender. Even if objects themselves are fully encapsulated – each object has full control of its own data – references or pointers to whole objects are equally meaningless in messages if the recipient is in a different address space. Emerald [20, 29] solves this problem by not including pointers in the language. ACT++ [21] and Concurrent C^{++} [16], which are both based on C^{++}, warn the programmer not to use references in messages to actors/processes on different nodes, but they do not enforce this restriction.

Traditionally, message passing constructs are divided into two categories: blocking and non-blocking (also called synchronous and asynchronous respectively). A blocking send blocks the sender – causing the sender's thread to be suspended – until the message is received and the message contents transferred into the address space of the recipient. A non-blocking send allows the sender to enter the message into a queue for transmittal, and then continue without waiting. Similarly, a blocking receive causes the recipient to wait until a message is delivered, and a non-blocking receive allows the input queue to be queried for a message without blocking the recipient. Blocking receive is common to most languages with explicit message passing, but both blocking and non-blocking forms of send are prevalent. Each is claimed to have its own advantages and disadvantages, which are discussed at some length in [16].

There are several advantages to blocking send. First, no buffering is needed. This can be an advantage because when there are buffers, they can become full, which can cause processes to block unpredictably when sending a message. Deadlock can result when a process becomes blocked before it can send a message of the specific type being awaited by the target process. Second, the use of blocking send guarantees the order of transmission of messages between a source/target pair. When sends are buffered, and subsequently processed asynchronously, such ordering is not necessarily guaranteed. The principal disadvantage cited for blocking send is that it limits concurrency within a program. When a process can continue without having to wait for a transmitted message to be delivered, there is clearly a greater amount of concurrency than if the process is blocked. Asynchronous message passing, however, offers no real advantage if the transmitting process has to wait anyway and allow the receiving process time to compute and return a result. Because of this, systems that support remote procedure call typically also support synchronous message passing. Machine architecture can also be a key factor in deciding the form of message passing to choose. Some architectures, such as transputer-based systems [19], implement blocking send at the hardware level. Other systems, such as the iPSC [5], provide system and hardware support for buffering.

3 Overview of Concurrent Object-Oriented Languages

We classify many of the existing concurrent object-oriented languages into four categories, according to how well the paradigms of OOP and concurrent programming are integrated. These categories are listed below in decreasing order of the level of integration of these paradigms.

1. Languages with Active Objects. In these languages the notions of *object* and *process* are unified into a single entity of an *active object* (i.e. object = process). In this class of languages the protection and message passing capabilities of an object are the basis of communication and synchronization. These languages naturally lend themselves to a distributed model of computation, hence are scal-

able. In most of these languages an object has exactly one thread of control. The application areas of these languages include artificial intelligence, systems, symbolic computations, and real-time applications. We identify two sub-categories of languages with active objects. The first sub-category includes languages in which an object is active as long as there are messages in its message queue. Most of the actor-based languages lie within this category (e.g. ABCL/1 [35], Act 1 [24], ACT^{++} [21]). Actor-based languages normally support fine grain parallelism. However, ABCL/1, and ACT^{++} allow for the capability of larger grain sizes. One way to achieve this is by using numbers, strings, lists, etc. as primitive data structures. Operation invocations to manipulate the above data structures and control structures such as if-then-else are also available as primitive facilities. Another way is to use passive objects in the construction of active objects. A passive object is an object with no thread of control whose identity should only be known to the containing active object. See Section 4.1 for a discussion of passive objects. Message passing in Actor-based languages is non-blocking. ACT^{++} is a C^{++}-based language.

The second sub-category is languages in which an object has an activity of its own and responds to messages at predetermined places during its execution. An example of such a language is the family of POOL languages [2, 3]. An object in a POOL language has an activity, called its *body*. A POOL object becomes active as soon as it is created. In POOL all objects are active (even primitive data structures are capable of responding to messages). POOL supports fine grain parallelism and blocking and non-blocking styles of message passing.

2. Languages with Parallel Method Invocations and Futures. Languages with active objects fully integrate OOP and concurrent programming paradigms by unifying the notions of a process and an object. We identify a second category of languages in which there is still a strong integration of the above mentioned paradigms, but concurrency is introduced by allowing asynchronous method invocations and not by unification of processes and objects. Synchronization and mutual exclusion is to some degree controlled by the programmer. These languages use future variables for receiving replies from asynchronous method calls. COOL [12] and ConcurrentSmalltalk [36] are two such examples. These languages allow multiple threads of control within an object. Both languages assume a shared memory model, and support medium to large grain parallelism. COOL is a C^{++}-based language.

3. Languages with Selectively Active Objects and Method Invocation by Caller's Thread. In this class of languages, the programmer is given the option of declaring some objects as active. An active object has its own thread of control. Non-active objects are visible to the outside world and they use the thread of the caller to advance their computation. An object may allow many threads of control to pass through it. These languages provide class abstractions, such as Monitor classes, to provide mutual exclusion. The multi-threaded nature of these languages usually demands a shared memory model. Examples of such languages are μC^{++} [11] and Emerald [20, 29]. μC^{++} is an extension of C^{++}.

4. Languages with Concurrency Orthogonal to OOP. In these languages there is little or no relationship between a process and an object. The designers of these languages consider OOP and concurrent programming as two orthogonal paradigms. They use the abstraction facilities of OOP to develop a library of classes for creation of processes, synchronization and mutual exclusion of data. Examples of such languages are Concurrent C^{++} [16] and PRESTO [8]. Both these languages support a shared memory model and medium to large grain parallelism. Concurrent C^{++} discourages programmers from using references so that programs can be ported to distributed memory machines. The compiler, however, does not enforce this. Both PRESTO and Concurrent C^{++} are C^{++}-based. Not strictly a language, PRESTO is a class library for the creation and management of threads. It is included here for completeness.

4 C^{++}-Based Concurrent Languages

In this section we first provide a summary of concurrent C^{++}-based languages. We then analyse their concurrency features.

4.1 Summary

ACT^{++} is an actor-based language. For more detail on actor models the reader is referred to [1, 33]. Concurrency is achieved by supporting active objects. It is a heterogeneous language, in the sense that it supports both active and passive objects. Instruction level concurrency, often found in actor languages, is not supported in ACT^{++}. The language supports medium to large grain parallelism. Communication between active objects is asynchronous or non-blocking. Synchronization and mutual exclusion is controlled by active objects. Hence ACT^{++} fully integrates the OOP and concurrent programming paradigms.

Primitive data and control structures of C^{++}, and passive objects are used in the construction of an active object. The identity of the passive objects should only be known to the containing active object (this is enforced only by convention). Passive objects do not have their own thread of control and use the thread of the active object containing them to advance their computation. ACT^{++} programmers use three classes – ACTOR, Mbox, and Cbox – to create actors (active objects), mail boxes for actors, and future variables for receiving replies, respectively.

Concurrency is created by either creating new actors with their own thread of control or by defining new behaviours for existing actors through the use of the *become* statement. In an actor-based language, the *become* facility is both a synchronization mechanism as well as a mechanism for changing the state of an actor (actor operations are side-effect free). Each behaviour of an actor has its own thread of control. An actor, after receiving a message, defines a replacement behaviour (i.e. a new actor) that may have a different message protocol and new values for its instance variables (called acquaintances in the actor model). An actor prevents the processing of other messages until the replacement behaviour is ready. The replacement behaviour proceeds with the processing of the next message in the mail queue.

COOL is an extension of C^{++} with additional constructs to create concurrency. COOL supports medium to large grain parallelism. Concurrency is created by declaring some *private* or *public* member functions as *parallel*. Method invocation of parallel methods is asynchronous and future variables are used to receive the results of the calls. Thread creation is at the time of method invocation. If a public member function requires exclusive access to an object, it is declared as *mutex*. The use of a mutex function may result in a deadlock. A mutex function executing on an object may need the result of another function executing on the same object. If the first function blocks on this object, a deadlock would occur. To avoid this, COOL has a *release* statement that can be used by a mutex function. The use of *release* blocks the mutex function and releases the object so that other functions can access it. Communication in COOL is through shared memory.

Besides mutex functions and future variables as synchronization mechanisms, COOL provides *locks* for synchronization at a finer granularity. The language does not support active objects, hence mutual exclusion and synchronization is to a greater degree the responsibility of the programmer. Leaving the responsibility of avoiding deadlocks to programmers through the use of lower level constructs like *release* is error prone. The language in its present implementation has not integrated some features of C^{++} (e.g. *friend*, *virtual* functions, *inheritance*) with the new concurrency constructs.

μC^{++} is an extension of C^{++} and supports co-routines, monitors, active objects (called tasks in μC^{++}), as well as regular, sequential objects. As such, in this language, objects are selectively active: the programmer generates concurrency in a program by choosing some objects to have *main* routines, i.e. active bodies. The assumed architecture is shared memory. Message passing is by method invocation, and may be thought of as blocking, in the sense that a calling thread may be blocked by virtue of mutual exclusion. The usual notion of block-

ing send – the sender is blocked until the message body is copied into the address space of the recipient – does not apply here, as it is the method arguments that comprise the message body, and these are stored only in the caller's stack frame. They are not copied (the shared memory model makes this possible). μC^{++} does, nonetheless, retain the flavor of message passing by virtue of an accept statement, which allows messages to be accepted selectively.

An object may have several threads in it at one time, but if the object is a monitor or a task, only one can be active at a time. When a task method is invoked, it is the caller's thread that enters the method. Mutual exclusion is achieved by blocking the task thread within the accept statement. The calling thread must remain in the method until the method is completed, usually by executing a return statement to return a value to the caller. Thus a μC^{++} method invocation is like a remote procedure call. Unlike Concurrent C^{++} transactions (another form of remote procedure call), the processing of the call can be deferred if required, by waiting on a condition queue. When a thread places itself on a queue to wait for a specified condition to be signaled, it releases exclusive access to the object, allowing another thread to proceed. To a certain extent, this capability makes up for the inability of the accept statement to examine actual arguments when determining which message to accept (a capability that is present in Concurrent C^{++}). It should be noted, however, that a deferred thread is not free to accomplish useful concurrent work during its deferral.

Concurrent C^{++} was developed at AT&T Bell Laboratories. It is an integration of AT&T's Concurrent C and C^{++} programming languages, both of which are upward-compatible supersets of C. Hence Concurrent C^{++} offers both the concurrency facilities of Concurrent C and the data abstraction facilities of C^{++}. Concurrency is achieved by creating many communicating processes. Concurrent C^{++} uses the term *transaction* for process interaction. Transactions may be synchronous (blocking) or asynchronous (non-blocking). Asynchronous transactions are one-way communications and return no results to the caller. Transactions may also be used for process synchronization. Processes can selectively accept messages by using an Ada-like *select* statement. Facilities such as priority specification and waiting for multiple events are also provided. Classes of C^{++} may be used to hide implementation details and enforce the visible external protocol of a process.

PRESTO is a class library for the creation and management of threads. Although it is not a language, as such, it is included in this paper for completeness. There are PRESTO classes for each concurrency concept, principally threads, locks, and monitors. Creating a thread object creates, within the thread object data area, the structures necessary for a thread's existence (e.g. a stack), although, notably, it does not start a new thread running – the thread *start* method must be called for this to happen. Similarly, creating a lock object allocates storage for the lock, which is then manipulated via lock methods (both spinning locks and relinquishing locks are available). Monitors are created and manipulated in the same fashion, which distinguishes PRESTO monitors from what is traditionally thought of as a monitor. In PRESTO, monitor routines are not physically associated with the monitor control object. The programmer must explicitly indicate that a monitor routine is being entered (via a call to an entry method) and when it is being exited (via the *exit* method).

The assumed architecture for PRESTO is shared memory; which is intended as a public domain threads library for shared memory multiprocessors. As such, it has no notion of message passing, even in the guise of C^{++} method invocation; communication happens solely through the use of shared variables. PRESTO objects are not active (in that they do not have bodies), but PRESTO can be used to implement a task class library to support objects with active bodies. Even in this case, however, the programmer must explicitly start the thread in a newly created task object. PRESTO, thus, comprises a set of low-level routines for shared-memory concurrency. Its principal advantage is high performance on such architectures.

4.2 Analysis

We now analyze the above languages with respect to the properties presented in Table 1.

We introduced four categories of concurrent languages in Section 3. We notice at least one C^{++}-based language in each category. Table 1 summarizes the characteristics of these languages.

We use the term caveat emptor to refer to languages that are implemented assuming a distributed memory model. Namely, objects/processes of these languages communicate by message passing and not by shared memory. Programmers, however, must follow a set of conventions, such as no use of references, in order for their programs to port to distributed memory machines. The compilers of these languages do not enforce these conventions. We now describe the table in more detail.

Assumed Architecture. Because of the intrinsic nature of C^{++} programs (most notably the lack of true encapsulization), most existing C^{++}-based concurrent languages assume a shared memory architecture. ACT^{++} and Concurrent C^{++} request that programmers follow a set of conventions when the underlying architecture has a distributed memory model. Relying on programmers to follow conventions is potentially dangerous and may lead to complicated and costly errors. The distributed memory model is a significant architecture that is rapidly becoming more popular. Hence it is advantageous for a concurrent C^{++}-based language to support such a model and enforce restrictions such as no use of references.

Message Passing. A concurrent object-oriented language must have a strong modelling capability for real world problems as well as maximum controlled parallelism. To achieve these goals, offering only a blocking style of message passing is probably too restrictive. Both blocking and non-blocking styles of message passing are necessary. It is also desirable to receive results from non-blocking communications, to allow a higher degree of parallelism. A caller requesting the services of another object can attend to other duties while the called object is processing the request. Future objects (variables) are commonly used to implement two-way non-blocking communications. As was mentioned in Section 2, the blocking style of message passing has many advantages, including not requiring buffering of messages – hence avoiding the associated overhead. Some of the advantages of non-blocking communication can be obtained by using future objects to provide the effects of asynchrony.

Active Objects. Languages with active objects offer a natural way of modelling real world concurrent objects. Message passing, synchronization, mutual exclusion, and creation of threads is all controlled at the object level. Programmers do not have to deal with new abstractions of processes, monitors, routines, tasks, etc. Such languages, however, may suffer from excessive overhead, in particular when parallelism is at a finer granularity.

Threads/Object. The number of threads in an active object has been the subject of much debate. Once again, if a concurrent language is to offer a powerful modelling capability, it must provide some level of concurrency within an object. Many real-world objects have internal concurrency. An uncontrolled level of nondeterminism, however, can lead to error-prone programs. A good compromise seems to be allowing objects to have multiple threads with only one thread active at a time. [34] classifies such objects as quasi-concurrent. This can be viewed as composing smaller objects to form a super object. Each object within a super object has its own thread, but only one thread is active within the super object at a time. The super object controls the activities of the threads. When an object is waiting on a call to another object, its thread is suspended and control is given to other objects within the super object.

Inheritance. Problems with incorporating inheritance within concurrent object-oriented languages have been noticed by many authors [10, 22, 28, 32, 34]. Briot and Yonezawa [10] point out that in many object-oriented languages, the relationship between a subclass and

	ACT^{++}	COOL	μC^{++}	Concurrent C^{++}	PRESTO
Category According to §3	1	2	3	4	4
Assumed Architecture	CE	SM	SM	CE	SM
Message Passing	Non-B	NA	Blocking[1]	B , Non-B	NA
Active Objects	yes	no	some	no	no
Threads/Object	$n \geq 0$	$n \geq 0$	0 or 1	NA	NA
Inheritance	yes	no	limited	NA	no

[1] μC^{++} assumes a shared memory model. See the Section on μC^{++} for more detail on the way μC^{++} handles message passing.

[2] CE: Caveat Emptor, SM: Shared Memory, B: Blocking, Non-B: Non-Blocking, NA: Not Applicable.

Table 1. Concurrency Features of C^{++}-Based Languages.

its superclass is *hard-wired*, and is usually implemented by pointers. This hard-wired relationship does not lend itself to distributed memory models. In fact, this is not a serious problem provided that all code can be copied in each processor. Furthermore, C^{++} does not use pointers to represent the relationship between a class and its subclass, hence this is not an issue in C^{++}. The problem of interference between synchronization mechanisms and inheritance is noticed by Kafura and Lee [22] as well as Tomlinson and Singh [32]. Tomlinson defines a synchronization mechanism to interfere with inheritance if local changes in the inheritance hierarchy (e.g. adding a new method to a subclass) require changes in the synchronization constraints elsewhere in the hierarchy. Due to this interference, many languages do not support inheritance or support only a limited inheritance.

Concurrent object-oriented languages use one of two ways to control concurrency: *centralized* and *decentralized* [22]. In the centralized approach one procedure controls the concurrency by indicating when it is ready to accept a particular message. Examples of such languages are the family of POOL languages and ABCL/1. In such languages the addition of a new method always requires changes in the procedure that controls concurrency. In the decentralized approach the concurrency control is distributed among methods. This may be achieved either by the use of critical sections or by objects changing their interface (interface control, using Kafura and Lee's terminology). Once again Kafura and Lee show that this approach to concurrency control either severely compromises other properties of object-oriented languages (e.g. encapsulation) or interferes with inheritance.

We describe this by showing how this problem occurs in ACT^{++}. Synchronization in ACT^{++} is handled through the use of the *become* statement. An actor, after receiving a message, defines a replacement behaviour and then proceeds with processing the message. The replacement behaviour has its own thread of control and proceeds in parallel with the original actor. Each replacement behaviour in ACT^{++} is described as a new class with its own protocol that describes the kinds of messages that it accepts.

Consider a class A with methods m1 and m2 and replacement behaviours A1 and A2. Assume in the body of methods m1 and/or m2 we use a *become* statement (i.e. *become*(A1) or *become*(A2)). Now suppose we want to extend A by having it accept a third method m3. One way to handle this is by introducing a new class B, that extends A by adding a new method m3. Any replacement behaviour of B that uses m3 is different from corresponding replacement behaviours in A, therefore we must describe new classes for these replacement behaviours. Assume B has replacement behaviours B1 and B2 that differs from A1 and A2 (i.e. they refer to method m3). Considering in the definition of class A, methods m1 and m2 explicitly refer to replacement behaviours A1 and A2, class B can not inherit methods m1 and m2 from class A.

Hence, concurrency control is interfering with inheritance. [22, 32] introduce additional abstraction facilities to aid the inheritance process in the context of languages that use interface control to synchronization.

COOL does not support inheritance in its present implementation. It uses the decentralized approach in controlling concurrency, hence adding inheritance requires solving the problems of interference between synchronization mechanisms and inheritance. Concurrent C^{++} does not support inheritance across processes. It uses the centralized approach in controlling the concurrency, hence inheritance between processes interferes with synchronization. μC^{++} supports inheritance in a limited way. While the usual inheritance rules apply to regular sequential objects, classes defined with extended types (co-routines, monitors and tasks) can inherit only from classes of the same type. Among tasks, when multiple bodies are to be inherited, the one selected to be the active body is the one declared the deepest in the class hierarchy. The programmer must define a new body to accommodate new methods – the existence of a new method does not block the inheritance of a previously defined body (which necessarily cannot have an accept statement for such new methods).

4.3 Research Issues and Concluding Remarks

Concurrent object-oriented programming is still in its infancy. Graceful integration of OOP and concurrent programming paradigms is the subject of on-going research. There are very few commercially available systems and very little, if any, performance benchmarks on the available systems. For a commercial concurrent system to be successful, it must provide efficiency, flexibility and simplicity.

There are many factors to be considered in introducing concurrency to C^{++}. First and foremost, one has to decide whether or not to ignore the natural fit between message passing and method invocation. Noting that communication at the hardware level in a distributed memory machine is via message passing, and that one of the major goals of object-oriented programming is to encapsulate objects (thus making them suitable for distribution), we feel that the message/method tie is too strong not to incorporate. That leaves the next major hurdle: how to achieve true encapsulation in C^{++}. In C^{++}, any object can access the private data of any other object in the same class. Furthermore, data members may be more visible still, even to every object in the case of public data members, or to every object in a class hierarchy in the case of protected data members. Even beyond that, pointers to internal data members may be passed from an object as a return result. Implementing any of these "features" necessitates a shared memory model. While there is nothing inherently wrong with the shared memory model, being tied to it inhibits portability (and possibly scalability).

A related issue is model "purity" versus overhead. In a pure, active object model, all objects are active, and all method invocations are messages. While such a model is easy to understand, and (assuming encapsulation) easy to distribute, there will inevitably be a large overhead associated with such fine-grained parallelism. Including passive objects in the model (and possibly monitor objects – in which methods are invoked as function calls) allows the active objects to be larger grained, thereby reducing overhead. This gives rise to another question: should passive objects also be distributable? If so, what is the meaning of the invocation of passive methods across processors? In other words, if an object does not have a thread, and can not use the thread of the caller, how can its methods be invoked and/or messages to it be processed?

We see the answers to some of these questions as follows. First, it seems there is a need for two "sorts" of objects in a concurrent C^{++}: distributable and non-distributable, where non-distributable objects are private to a processor, and are known only to objects on a given processor. If dynamic migration of distributable objects is to be achieved, then non-distributable objects should be known only within an object cluster (possibly of unitary size) that is migrated as a whole. A solution to distributable passive objects could be to define them as being pseudo-active, with a default body. Their methods would be invoked by available threads on the node of residence,

but without the overhead associated with flexible message acceptance.

Extending code sharing and reuse facilities of object-oriented languages to a concurrent environment (which would include synchronization code) is a difficult problem and the subject of much debate.

Acknowledgment. The authors gratefully acknowledge the many helpful comments and suggestions of Professor Michael Jenkins of Queen's University.

5 References

1. G. Agha, ACTORS: A Model of Concurrent Computation in Distributed Systems, MIT Press, 1986.

2. P. America, POOL-T: A Parallel Object-Oriented Language, in Object-Oriented Concurrent Programming, edited by A. Yonezawa and M. Tokoro, The MIT Press, 1987.

3. P. America, A Parallel Object-Oriented Language with Inheritance and Subtyping, ECOOP/OOPSLA'90, 161-168.

4. G. R. Andrews and F. Schneider, Concepts and Notation for Concurrent Programming, Computing Surveys, Vol. 15, No. 1, March 1983, 3-43.

5. R. Arlauskas, iPSC/2System: A Second Generation Hypercube, The Third Conference on Hypercube Concurrent Computers and Applications, 1988, 38-42.

6. H. Bal, J. G. Steiner and A. S. Tanenbaum, Programming Languages for Distributed Computing Systems, ACM Computing Surveys, Vol.21, No. 3, 1989, 262-322.

7. H.E. Bal, The Shared Data-object Model as a Paradigm for Programming Distributed Systems. Ph.D. Thesis, Dept. Of Mathematics and Computer Science, Vrije Universiteit, Amsterdam, the Netherlands, 1989.

8. B. Bershad, E. D. Lazowska and H. M. Levy, PRESTO: A System for Object-Oriented Parallel Programming, Software-Practice and Experience, Vol. 18(8), 713-732, August 1988.

9. S. Bhatt, M. Chen, C.Y. Lin and P. Liu, Abstractions for Parallel N-body Simulations, DCS/TR-895, Yale University, 1992.

10. J.P. Briot and A. Yonezawa, Inheritance and Synchronization in Concurrent OOP, ECOOP'87, 32-40.

11. P.A. Buhr, G. Ditchfield, R. A. Stroobosscher, B. M. Younger, and C. R. Zarnke, μC^{++}:
Concurrency in the Object-Oriented Language C^{++}, Software-Practice and Experience, 22(2), 1992, 137-172.

12. R. Chandra, A. Gupta and J. Hennessy, COOL: a Language for Parallel Programming, in Languages and Compilers for Parallel Computing edited by D. Gelernter, A. Nicolau, D. Padua, The MIT Press, 1990.

13. B. Cox, Object-Oriented Programming: An Evolutionary Approach, Addison-Wesley, Reading, MA, 1986.

14. O.J. Dahl and K. Nygaard, Simula - An Algol-based Simulation Language, Comm. of ACM, 9:9 (Sept. 1966), 671-678.

15. T.W. Doeppner Jr., and Alan J. Gebele, "C^{++} on a parallel machine," Report CS-87-26, Department of Computer Science, Brown University, November 1987.

16. N.H. Gehani and W.D. Roome, Concurrent C^{++}: Concurrent Programming with Class(es), Software-Practice and Experience, Vol. 18(12), 1988, 1157-1177.

17. A. Goldberg and D. Robson, Smalltalk-80: The Language and its Implementation, Addison Wesley, MA, 1983.

18. R.C. Holt, T. Stanhope and G. Lausman, Object Oriented Computing: Looking Ahead to the Year 2000, IBM TR-74.060, also in the Proceedings of the 1991 CAS Conference, IBM Canada Ltd. Laboratory, October 1991.

19. M. Homewood, D. May, D. Shepherd and R. Shepherd, The IMS T800 Transputer IEEE Micro, October 1987.

20. E. Jul, H. Levy, N. Hutchinson and A. Black, Fine-Grained Mobility in the Emerald System, ACM Trans. on Computer Systems, 6(1), 1988, 109-133.

21. D. Kafura and K.H. Lee, *ACT++*: Building a Concurrent *C++* with Actors, JOOP, Vol. 3, No. 1, 1990, 25-37.

22. D. Kafura and K.H. Lee, Inheritance in Actor Based Concurrent Object-Oriented Languages, The Computer Journal, Vol. 32, No. 4, 1989, 297-304.

23. W. Kim and F.H. Lochovsky, Object-Oriented Concepts, Databases, and Applications, ACM Press, 1989.

24. H. Lieberman, Concurrent Object-Oriented Programming in Act 1, OOPSLA'86, 258-268.

25. J. Lim and R.E. Johnson, The Heart of Object-Oriented Concurrent Programming, SIGPLAN Notices, October 1988, 165-167.

26. B. Meyer, Object-Oriented Software Construction, Prentice Hall, 1988.

27. O. Nierstrasz, A Survey of Object-Oriented Concepts, in [28].

28. M. Papathomas, Language Design Rationale and Semantic Framework for Concurrent Object-Oriented Programming, Ph.D. Thesis, University of Geneva, 1992

29. K.R. Rajendra, E. Temprero and H. M. Levy, Emreld: A General-Purpose Programming Language, Software-Practice and Experience, Vol. 21(1), January 1991, 91-118.

30. B. Stroustrup, The *C++* Programming Language (2nd Edition), Addison-Wesley, 1991.

31. M. Stumm and S. Zhou, Algorithms Implementing Distributed Shared Memory, IEEE Computer, May 1990, 54-64.

32. C. Tomlinson and V. Singh, Inheritance and Synchronization with Enabled-Sets, OOPSLA'89, 103-113.

33. C. Tomlinson and M. Scheevel, Concurrent Object-Oriented Languages, in [28].

34. P. Wegner, Dimensions of Object-Based Language Design, OOPSLA'87, 168-182.

35. A. Yonezawa, J.P. Briot and E. Shibayama, Object-Oriented Concurrent Programming in ABCL/1, OOPSLA'86, 258-268.

36. Y. Yokote and M. Tokoro, Concurrent Programming in ConcurrentSmalltalk, in "Object-Oriented Programming", edited by A. Yonezawa and Mario Tokoro, 1987.

About the authors

Eshrat Arjomandi is an Associate Professor of Computer Science at York University. Her research interests are in the area of concurrent object-oriented languages.

Both her Masters and PhD degrees are from the University of Toronto. She can be reached at the Department of Computer Science, York University, North York, Ontario, M3J-1P3. Her email address is eshrat@cs.yorku.ca.

William O'Farrell is a Research Staff Member at the Centre for Advanced Studies, IBM Canada Laboratory. His research interests are in the area of parallel computation, object oriented concurrent systems, and programming languages. He joined IBM in May of 1991, after completing his Ph.D. at Syracuse University. His Masters degree is from Queens University at Kingston. He can be reached at Centre for Advanced Studies, IBM Canada Ltd., B2/894, 895 Don Mills Rd., North York, Ontario M3C 1W3. His email address is billo@torolab2.vnet.ibm.com.